Baltimore MTA Transit Simulator

A Real-Data Transit Network Simulator

Demonstrating Core Data Structures & Algorithms

Course: COSC 320 - Algorithm Design

Student: Aayush Shrestha

University: Morgan State University

Date: March 2026

Language: Python 3.13 with Rich terminal UI

GitHub: github.com/theaayushstha1/transit

This project implements a transit network simulator using real Baltimore MTA data (Light Rail and Metro SubwayLink, 37 stations). It demonstrates graph traversal (DFS, BFS), shortest path (Dijkstra's), sorting (Merge Sort, Quick Sort), searching (Binary, Linear), and queue/stack data structures through an interactive terminal interface with animated algorithm visualizations.

1. System Design & Architecture

The simulator is organized into five modules following separation of concerns. Each module has a distinct responsibility, making the codebase maintainable and each algorithm independently testable.

1.1 Directory Structure

```
 baltimore-mta-simulator/
|-- main.py                  # Entry point, menu loop
|-- models/
|   |-- station.py           # Station node class
|   |-- passenger.py         # Passenger class
|   +-- transit_network.py   # Graph (adjacency list)
|-- algorithms/
|   |-- traversal.py         # DFS + BFS
|   |-- shortest_path.py     # Dijkstra's
|   |-- sorting.py           # Merge Sort + Quick Sort
|   +-- searching.py         # Binary + Linear Search
|-- simulation/
|   |-- undo_manager.py      # Stack-based undo
|   +-- passenger_sim.py     # Queue-based boarding
|-- data/
|   +-- baltimore_mta.py     # Real MTA station data
+-- ui/
    |-- display.py           # Rich tables, panels, menus
    +-- visualizer.py        # Algorithm animations
```

1.2 Design Decisions

Decision	Choice	Reasoning
Graph repr.	Adjacency list (dict of dicts)	O(1) edge lookup, natural for sparse graphs
Edge weights	Travel time in minutes	Enables Dijkstra, maps to real MTA data
Shared stations	Dedup by name	Lexington Market on both lines = 1 node
UI library	Python Rich	Colored tables, live animations, zero web deps
Undo pattern	Command pattern + inverse	Clean stack demo, _raw_ methods skip undo
Sorting key	Station capacity	Numeric field with varied values for comparison

2. Data Structures

2.1 Graph (Adjacency List)

The transit network is represented as a weighted undirected graph using an adjacency list. The implementation uses a dictionary of dictionaries: adj_list[station_id] maps to {neighbor_id: weight_in_minutes}. This provides O(1) edge lookup and insertion, which is optimal for the sparse graph topology of a transit network (37 nodes, 36 edges).

```
class TransitNetwork:
  def __init__(self):
      self.stations: dict[int, Station] = {}
      self.adj_list: dict[int, dict[int, float]] = {}
      self.undo_stack: list = []

  def add_connection(self, id1, id2, weight):
      self.adj_list[id1][id2] = weight  # undirected:
      self.adj_list[id2][id1] = weight  # store both
```

2.2 Queue (FIFO - Passenger Boarding)

Each station maintains a passenger queue using collections.deque. Passengers are enqueued (append) when generated and dequeued (popleft) during boarding, demonstrating FIFO behavior. The deque provides O(1) for both operations, unlike a list which would be O(n) for popleft due to element shifting.

```
 # Enqueue: passenger joins the line
station.passenger_queue.append(passenger)

# Dequeue: board in FIFO order
while queue and len(boarded) < capacity:
    passenger = station.passenger_queue.popleft()
```

2.3 Stack (LIFO - Undo System)

The undo system uses a Python list as a stack (append/pop). Every graph mutation (add station, remove station, add/remove connection) pushes its inverse action onto the stack. The UndoManager pops actions and applies the inverse using internal _raw_ methods that bypass the undo stack to prevent infinite loops.

```
 # Public method pushes to undo stack
def add_station(self, station):
    self._raw_add_station(station)
    self.undo_stack.append({"action": "add_station", ...})

# Undo pops and applies inverse via _raw_ method
def undo(self):
    action = self.network.undo_stack.pop()
    if action["action"] == "add_station":
        self.network._raw_remove_station(action["station"].id)
```

2.4 Priority Queue (Min-Heap - Dijkstra)

Dijkstra's algorithm uses Python's heapq module as a min-heap priority queue. Each entry is a (distance, station_id) tuple. heappush and heappop both operate in O(log n) time, giving Dijkstra's algorithm its O((V+E) log V) time complexity.

3. Algorithm Implementations

3.1 Depth-First Search (DFS)

DFS is implemented recursively. It explores as deeply as possible along each branch before backtracking. The algorithm maintains a visited set to avoid cycles and a path list that grows/shrinks via append/pop as it explores/backtracks. It also records the visit order for the animated visualization.

```
 def _dfs_helper(current: int) -> bool:
   visited.add(current)
   visit_order.append(current)
   path.append(current)
   if current == goal_id:
       return True
   for neighbor_id, _ in sorted(network.get_neighbors(current)):
       if neighbor_id not in visited:
           if _dfs_helper(neighbor_id):
               return True
   path.pop()  # backtrack
   return False
```

3.2 Breadth-First Search (BFS)

BFS uses a collections.deque as a queue. It explores all neighbors at the current depth before moving to the next level. BFS guarantees the shortest path in an unweighted graph (fewest edges). Each queue entry stores the full path to that node for easy path reconstruction.

```
 visited = {start_id}
queue = deque([(start_id, [start_id])])
while queue:
    current, path = queue.popleft()
    if current == goal_id:
        return path
    for neighbor_id, _ in sorted(network.get_neighbors(current)):
        if neighbor_id not in visited:
            visited.add(neighbor_id)
            queue.append((neighbor_id, path + [neighbor_id]))
```

3.3 DFS vs BFS Comparison

When finding a route from Hunt Valley to Johns Hopkins Hospital, DFS visited all 37 stations (exploring every branch before finding the goal), while BFS visited only 23 (expanding level-by-level and stopping as soon as the goal is reached). Both found a 17-station path in this case because the graph is largely linear. In a denser graph, BFS would find the shorter path while DFS might find a longer one.

Metric	DFS (Recursive)	BFS (Queue)
Path Length	17 stations	17 stations
Nodes Visited	37	23
Optimal (unweighted)?	Not guaranteed	Yes
Time Complexity	O(V + E)	O(V + E)
Space Complexity	O(V) recursion stack	O(V) queue

3.4 Dijkstra's Shortest Path Algorithm

Dijkstra's algorithm finds the shortest weighted path using a min-heap priority queue. It greedily selects the unvisited node with the smallest known distance, then relaxes its neighbors. The path from Hunt Valley to Johns Hopkins Hospital traverses 17 stations in 37 minutes, transferring from Light Rail to Metro at Lexington Market.

```
while pq:
  current_dist, current = heapq.heappop(pq)
  if current in visited: continue
  visited.add(current)
  if current == end_id: break
  for neighbor_id, weight in network.get_neighbors(current):
      new_dist = current_dist + weight
      if new_dist < distances[neighbor_id]:
          distances[neighbor_id] = new_dist
          previous[neighbor_id] = current
          heapq.heappush(pq, (new_dist, neighbor_id))
```

3.5 Merge Sort

Merge Sort divides the array in half recursively, then merges the sorted halves. It is stable (preserves relative order of equal elements) and always runs in O(n log n) time, but requires O(n) extra space for the merge step. On our 37 stations sorted by capacity, Merge Sort used 140 comparisons.

```
def _merge(left, right):
  result = []
  i = j = 0
  while i < len(left) and j < len(right):
      comparisons[0] += 1
      if key(left[i]) <= key(right[j]):
          result.append(left[i]); i += 1
      else:
          result.append(right[j]); j += 1
  result.extend(left[i:])
  result.extend(right[j:])
  return result
```

3.6 Quick Sort

Quick Sort selects a pivot (last element), partitions the array so elements smaller than the pivot go left and larger go right, then recurses on both halves. It is in-place (O(log n) stack space) but unstable and has O(n^2) worst-case on already sorted input. On our 37 stations, Quick Sort used 188 comparisons (more than Merge Sort's 140, as expected for this data distribution).

3.7 Sorting Comparison

Metric	Merge Sort	Quick Sort
Comparisons (n=37)	140	188
Time (measured)	~35 us	~23 us
Time Complexity	O(n log n) always	O(n log n) avg / O(n^2) worst
Space Complexity	O(n) extra	O(log n) stack
Stable?	Yes	No
In-place?	No	Yes

Quick Sort was faster in practice despite more comparisons because it is in-place (better cache locality, no memory allocation for merge buffers). Merge Sort's advantage is guaranteed O(n log n) worst case and stability.

3.8 Binary Search vs Linear Search

Binary Search requires a sorted array and eliminates half the remaining elements each step (O(log n)). Linear Search scans sequentially (O(n)). When searching for capacity 200 among 37 sorted stations, Binary Search found it in 3 comparisons while Linear Search needed 20.

Metric	Binary Search	Linear Search
Comparisons (target=200)	3	20
Time Complexity	O(log n)	O(n)
Requires Sorted?	Yes	No

4. Algorithm Complexity Analysis

The following table summarizes the theoretical time and space complexity of every operation implemented in the simulator. These are verified empirically by the performance dashboard, which runs live benchmarks.

Operation	Time Complexity	Space Complexity
Add Station	O(1)	O(1)
Remove Station	O(V + E)	O(1)
Add Connection	O(1)	O(1)
Remove Connection	O(1)	O(1)
DFS Traversal	O(V + E)	O(V) recursion stack
BFS Traversal	O(V + E)	O(V) queue
Dijkstra's Algorithm	O((V+E) log V)	O(V)
Merge Sort	O(n log n) always	O(n) extra
Quick Sort	O(n log n) avg / O(n^2) worst	O(log n) stack
Binary Search	O(log n)	O(1)
Linear Search	O(n)	O(1)
Stack Push/Pop	O(1)	O(1) per action
Queue Enq/Deq	O(1)	O(1)

4.1 Empirical Benchmarks

All algorithms were benchmarked on the full 37-station, 36-edge Baltimore MTA network. Timings use time.perf_counter() for microsecond precision. Sorting and searching algorithms also track exact comparison counts.

Algorithm	Measured Time	Comparisons
DFS (Hunt Valley to JHH)	~18 us	-
BFS (Hunt Valley to JHH)	~14 us	-
Dijkstra (Hunt Valley to JHH)	~12 us	-
Merge Sort (37 stations)	~35 us	140
Quick Sort (37 stations)	~23 us	188
Binary Search (target=200)	~0.3 us	3
Linear Search (target=200)	~1.2 us	20

Key observations: (1) BFS outperforms DFS on this graph because it visits fewer nodes by expanding level-by-level. (2) Dijkstra is the fastest graph algorithm because it terminates early when the target is reached. (3) Quick Sort is faster

than Merge Sort in practice due to cache-friendly in-place operations, despite more comparisons. (4) Binary Search uses 3 comparisons vs Linear's 20, confirming O(log n) vs O(n).

5. Baltimore MTA Transit Data

The simulator uses real Baltimore MTA data for two transit lines. Station names, line assignments, and approximate travel times are based on the actual MTA system map.

5.1 Light Rail (24 stations)

Runs from Hunt Valley in the north through downtown Baltimore (Penn Station, Camden Yards) to Cromwell/Glen Burnie in the south, with a spur to BWI Airport branching at Linthicum. The line has 23 connections with travel times of 2-5 minutes.

5.2 Metro SubwayLink (13 stations)

Runs from Owings Mills in the northwest through Mondawmin and downtown to Johns Hopkins Hospital in the east. The line has 13 connections with travel times of 2-4 minutes.

5.3 Transfer Point: Lexington Market

Lexington Market appears on both lines and is stored as a single node in the graph with 4 connections (2 Light Rail neighbors: Centre Street and Convention Center; 2 Metro neighbors: State Center and Charles Center). This creates the cross-line routing that makes Dijkstra's algorithm interesting: the shortest path from Hunt Valley (Light Rail) to Johns Hopkins Hospital (Metro) naturally routes through Lexington Market, demonstrating a real-world transfer.

5.4 Network Statistics

Property	Value
Total stations	37
Total connections	36
Light Rail stations	24 (including shared)
Metro stations	14 (including shared)
Transfer stations	1 (Lexington Market)
Min travel time	2 minutes
Max travel time	5 minutes (Linthicum to BWI)
Longest shortest path	37 min (Hunt Valley to JHH)

6. User Interface & Visualization

The simulator uses the Python Rich library for a polished terminal UI. All output is styled with colored text, formatted tables, bordered panels, and tree views. The Rich Live display enables real-time animated visualizations.

6.1 Interactive Menu

The main menu provides 10 options covering all functionality: network viewing, station/connection management, DFS vs BFS comparison, Dijkstra shortest path, passenger boarding simulation, sorting comparison, searching comparison,

performance dashboard, undo, and exit. Users can select stations by ID number or by typing the station name (case-insensitive matching).

6.2 Algorithm Animations

DFS and BFS traversals are animated using Rich Live display. Each step shows a table of all stations with color-coded status: white (unvisited), yellow (currently exploring), green (on the final path), and dim gray (visited but not on path). This makes the behavioral difference between DFS and BFS immediately visible: DFS goes deep then backtracks, while BFS expands outward level by level. The animation runs at ~6 frames per second, fast enough to convey behavior without dragging during a demo.

6.3 Passenger Boarding Animation

The boarding simulation animates a progress bar that fills as passengers board one by one, showing the FIFO queue in action. The display updates in real-time inside a bordered panel with station info, capacity, boarded count, and remaining queue count. For example, generating 50 passengers at Camden Yards with a train capacity of 30 boards exactly 30 in FIFO order, leaving 20 in the queue.

6.4 Dijkstra Path Display

Dijkstra results are shown as a visual chain with travel times on each edge: Station A --3min--> Station B --2min--> Station C. The panel also displays total travel time, number of stations, and number of line transfers. Cross-line routes automatically detect when the active transit line changes, counting the transfer at shared stations like Lexington Market.

6.5 Performance Dashboard

The performance dashboard runs live benchmarks on all algorithms and displays two tables: a theoretical complexity table (Big-O for time and space) and an empirical benchmarks table with measured execution time and comparison counts. This bridges theory and practice, showing that the measured performance matches asymptotic predictions.

7. Conclusion

This project demonstrates the practical application of core data structures and algorithms to a real-world transit network. By using actual Baltimore MTA station data rather than abstract examples, the project grounds algorithmic concepts in a tangible system that students and professors can relate to.

Key takeaways from the implementation:

1. Graph representation: An adjacency list (dict of dicts) provides O(1) edge operations and naturally models the sparse connectivity of a transit network.
2. DFS vs BFS: BFS guarantees the shortest unweighted path and typically visits fewer nodes, while DFS's deep exploration is visible in the animation. Both run in O(V+E) time.
3. Dijkstra's algorithm: The min-heap priority queue enables weighted shortest path in O((V+E) log V). Cross-line routing through shared stations demonstrates real graph connectivity.
4. Sorting: Merge Sort's guaranteed O(n log n) and stability comes at the cost of O(n) space. Quick Sort's in-place operation gives better cache performance despite O(n^2) worst case.
5. Searching: Binary Search's O(log n) dramatically outperforms Linear Search's O(n), confirmed empirically with 3 vs 20 comparisons on 37 stations.
6. Queue/Stack: The deque-based passenger queue and list-based undo stack demonstrate fundamental FIFO/LIFO patterns with O(1) operations.

The Rich terminal UI elevates the project beyond a standard command-line implementation, with animated algorithm visualizations that make abstract concepts tangible. The performance dashboard bridges theory and practice by displaying Big-O complexity alongside measured benchmarks, reinforcing that algorithmic analysis predicts real-world behavior.

Appendix: Screenshots

The following screenshots show actual terminal output from the simulator, captured using Rich's SVG export and converted to PNG.

Main Menu

```
Main Menu

Loaded Baltimore MTA network:        37 stations,      36 connections

┌──────────────
```

Network Map - Real Baltimore MTA Data

Network Map

```
Baltimore MTA Network
□──Light Rail      (23 stations)
 |   □──Hunt Valley      (ID: 1, Cap: 200)
 |    |    └──-> Pepper Road (3 min)
 |   □──Pepper Road    (ID: 2, Cap: 120)
 |    |    □──-> Hunt Valley (3 min)
 |    |    └──-> McCormick Road (2 min)
 |   □──McCormick Road    (ID: 3, Cap: 100)
 |    |    □──-> Pepper Road (2 min)
 |    |    └──-> Gilroy Road (2 min)
 |   □──Gilroy Road      (ID: 4, Cap: 80)
 |    |    □──-> McCormick Road (2 min)
 |    |    └──-> Lutherville (3 min)
 |   □──Lutherville        (ID: 5, Cap: 150)
 |    |    □──-> Gilroy Road (3 min)
 |    |    └──-> Falls Road (2 min)
 |   □──Falls Road      (ID: 6, Cap: 180)
 |    |    □──-> Lutherville (2 min)
 |    |    └──-> Mt Washington (3 min)
 |   □──Mt Washington      (ID: 7, Cap: 160)
 |    |    □──-> Falls Road (3 min)
 |    |    └──-> Cold Spring Lane (2 min)
 |   □──Cold Spring Lane        (ID: 8, Cap: 140)
 |    |    □──-> Mt Washington (2 min)
 |    |    └──-> Woodberry (2 min)
 |   □──Woodberry    (ID: 9, Cap: 130)
 |    |    □──-> Cold Spring Lane (2 min)
 |    |    └──-> North Avenue (3 min)
 |   □──North Avenue      (ID: 10, Cap: 250)
 |    |    □──-> Woodberry (3 min)
 |    |    └──-> Penn Station (2 min)
 |   □──Penn Station      (ID: 11, Cap: 400)
 |    |    □──-> North Avenue (2 min)
 |    |    └──-> University of Baltimore (2 min)
 |   □──University of Baltimore              (ID: 12, Cap: 200)
 |    |    □──-> Penn Station (2 min)
 |    |    └──-> Centre Street (2 min)
 |   □──Centre Street        (ID: 13, Cap: 200)
 |    |    □──-> University of Baltimore (2 min)
 |    |    └──-> Lexington Market (2 min)
 |   □──Convention Center        (ID: 15, Cap: 300)
 |    |    □──-> Lexington Market (2 min)
 |    |    └──-> Camden Yards (2 min)
 |   □──Camden Yards    (ID: 16, Cap: 500)
 |    |    □──-> Convention Center (2 min)
 |    |    └──-> Hamburg Street (2 min)
 |   □──Hamburg Street      (ID: 17, Cap: 180)
 |    |    □──-> Camden Yards (2 min)
 |    |    └──-> Westport (3 min)
 |   □──Westport    (ID: 18, Cap: 120)
 |    |    □──-> Hamburg Street (3 min)
 |    |    └──-> Cherry Hill (2 min)
 |   □──Cherry Hill        (ID: 19, Cap: 150)
 |    |    □──-> Westport (2 min)
 |    |    └──-> Patapsco (3 min)
 |   □──Patapsco    (ID: 20, Cap: 130)
 |    |    □──-> Cherry Hill (3 min)
 |    |    └──-> Linthicum (2 min)
 |   □──Linthicum      (ID: 21, Cap: 120)
 |    |    □──-> Patapsco (2 min)
 |    |    □──-> Ferndale (2 min)
 |    |    └──-> BWI Airport (5 min)
 |   □──Ferndale    (ID: 22, Cap: 100)
 |    |    □──-> Linthicum (2 min)
 |    |    └──-> Cromwell / Glen Burnie (3 min)
 |   □──Cromwell / Glen Burnie          (ID: 23, Cap: 160)
```

All 37 Stations

All Stations

ID	Name	Line	Capacity	Connections
1	Hunt Valley	Light Rail	200	1
2	Pepper Road	Light Rail	120	2
3	McCormick Road	Light Rail	100	2
4	Gilroy Road	Light Rail	80	2
5	Lutherville	Light Rail	150	2
6	Falls Road	Light Rail	180	2
7	Mt Washington	Light Rail	160	2
8	Cold Spring Lane	Light Rail	140	2
9	Woodberry	Light Rail	130	2
10	North Avenue	Light Rail	250	2
11	Penn Station	Light Rail	400	2
12	University of Baltimore	Light Rail	200	2
13	Centre Street	Light Rail	200	2
14	Lexington Market	Both	350	4
15	Convention Center	Light Rail	300	2
16	Camden Yards	Light Rail	500	2
17	Hamburg Street	Light Rail	180	2
18	Westport	Light Rail	120	2
19	Cherry Hill	Light Rail	150	2
20	Patapsco	Light Rail	130	2
21	Linthicum	Light Rail	120	3
22	Ferndale	Light Rail	100	2
23	Cromwell / Glen Burnie	Light Rail	160	1
24	BWI Airport	Light Rail	300	1
25	Owings Mills	Metro	300	1
26	Old Court	Metro	200	2
27	Milford Mill	Metro	180	2
28	Reisterstown Plaza	Metro	220	2
29	Rogers Avenue	Metro	160	2
30	West Cold Spring	Metro	140	2
31	Mondawmin	Metro	250	2
32	Penn North	Metro	200	2
33	Upton	Metro	180	2
34	State Center	Metro	220	2
35	Charles Center	Metro	300	2
36	Shot Tower	Metro	220	2
37	Johns Hopkins Hospital	Metro	280	1

Dijkstra's Shortest Path - Cross-Line Routing

```
Dijkstra Shortest Path
Shortest Path: Hunt Valley -> Johns Hopkins Hospital

Hunt Valley --3min--> Pepper Road --2min--> McCormick Road --2min--> Gilroy Road
--3min--> Lutherville --2min--> Falls Road --3min--> Mt Washington --2min--> Cold
Spring Lane --2min--> Woodberry --3min--> North Avenue --2min--> Penn Station
--2min--> University of Baltimore --2min--> Centre Street --2min--> Lexington Market
--2min--> Charles Center --2min--> Shot Tower --3min--> Johns Hopkins Hospital

Total travel time: 37 minutes
Stations: 17 Transfers: 1

Dijkstra completed in 16.54 us
```

DFS vs BFS Comparison

DFS vs BFS Comparison

DFS vs BFS Comparison

Metric	DFS (Recursive)	BFS (Queue)
Path Found?	Yes	Yes
Path Length	17 stations	17 stations
Nodes Visited	37	23
Time	19.04 us	14.67 us
Optimal? (unweighted)	Not guaranteed	Yes
Data Structure	Recursion stack	Queue (deque)

DFS path: Hunt Valley -> Pepper Road -> McCormick Road -> Gilroy Road -> Lutherville -> Falls Road -> Mt Washington -> Cold Spring Lane -> Woodberry -> North Avenue -> Penn Station -> University of Baltimore -> Centre Street -> Lexington Market -> Charles Center -> Shot Tower -> Johns Hopkins Hospital

BFS path: Hunt Valley -> Pepper Road -> McCormick Road -> Gilroy Road -> Lutherville -> Falls Road -> Mt Washington -> Cold Spring Lane -> Woodberry -> North Avenue -> Penn Station -> University of Baltimore -> Centre Street -> Lexington Market -> Charles Center -> Shot Tower -> Johns Hopkins Hospital

Merge Sort vs Quick Sort

Merge Sort vs Quick Sort

Sorting 37 stations by capacity ...

Sorting Comparison: Merge Sort vs Quick Sort

Metric	Merge Sort	Quick Sort
Comparisons	140	188
Time	39.42 us	23.63 us
Time Complexity	O(n log n) always	O(n log n) avg
Space Complexity	O(n) extra	O(log n) stack
Stable?	Yes	No

Stations Sorted by Capacity

Rank	Name	Capacity
1	Gilroy Road	80
2	McCormick Road	100
3	Ferndale	100
4	Pepper Road	120
5	Westport	120
6	Linthicum	120
7	Woodberry	130
8	Patapsco	130
9	Cold Spring Lane	140
10	West Cold Spring	140
11	Lutherville	150
12	Cherry Hill	150
13	Mt Washington	160
14	Cromwell / Glen Burnie	160
15	Rogers Avenue	160
16	Falls Road	180
17	Hamburg Street	180
18	Milford Mill	180
19	Upton	180
20	Hunt Valley	200
21	University of Baltimore	200
22	Centre Street	200
23	Old Court	200
24	Penn North	200
25	Reisterstown Plaza	220
26	State Center	220
27	Shot Tower	220
28	North Avenue	250
29	Mondawmin	250
30	Johns Hopkins Hospital	280
31	Convention Center	300
32	BWI Airport	300
33	Owings Mills	300
34	Charles Center	300
35	Lexington Market	350
36	Penn Station	400
37	Camden Yards	500

Binary Search vs Linear Search

Binary vs Linear Search

Search Comparison: Target Capacity = 200

Metric	Binary Search	Linear Search
Found	Old Court	Hunt Valley
Comparisons	3	20
Time	1.37 us	1.33 us
Time Complexity	O(log n)	O(n)
Requires Sorted?	Yes	No

Passenger Boarding Simulation (Queue/FIFO)

```
Passenger Boarding

Generated  50 passengers at Camden Yards

┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                                   Passenger Boarding                                                         │
│                                                                                                                              │
│   Station: Camden Yards                                                                                                      │
│  Passengers waiting: 50                                                                                                      │
│  Train capacity: 30                                                                                                          │
│                                                                                                                              │
│  Boarding:      ##############################      30/30 boarded                                                            │
│  Remaining in queue: 20                                                                                                      │
│                                                                                                                              │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

Undo System (Stack/LIFO)

```
Undo System

Step  1:  Add new station        'Morgan State'
   Added:   Station   ( 38: Morgan State      )
 Network now has        38  stations

Step  2:  Undo the addition
   Undid: added station          'Morgan State'
 Network back to        37  stations
```

Performance Dashboard - Theory + Benchmarks

Performance Dashboard

Running benchmarks ...

Algorithm Complexity (Theory)

Operation	Time Complexity	Space Complexity
Add Station	O(1)	O(1)
Remove Station	O(V + E)	O(1)
Add Connection	O(1)	O(1)
Remove Connection	O(1)	O(1)
DFS Traversal	O(V + E)	O(V) recursion stack
BFS Traversal	O(V + E)	O(V) queue
Dijkstra's Algorithm	O((V+E) log V)	O(V)
Merge Sort	O(n log n) always	O(n) extra
Quick Sort	O(n log n) avg / O(n^2) worst	O(log n) stack
Binary Search	O(log n)	O(1)
Linear Search	O(n)	O(1)
Stack Push/Pop (Undo)	O(1)	O(1)
Queue Enq/Deq (Passengers)	O(1)	O(1)

Empirical Benchmarks (n=37 stations, 36 edges)

Algorithm	Measured Time	Comparisons
DFS	17.50 us	-
BFS	12.04 us	-
Dijkstra	11.12 us	-
Merge Sort	35.33 us	140
Quick Sort	22.42 us	188
Binary Search	0.29 us	1
Linear Search	1.04 us	16

References

[1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.

[2] Baltimore MTA. Maryland Transit Administration System Map. mta.maryland.gov

[3] Python Software Foundation. collections.deque, heapq documentation. docs.python.org

[4] Will McGugan. Rich: Python library for rich text in the terminal. github.com/Textualize/rich